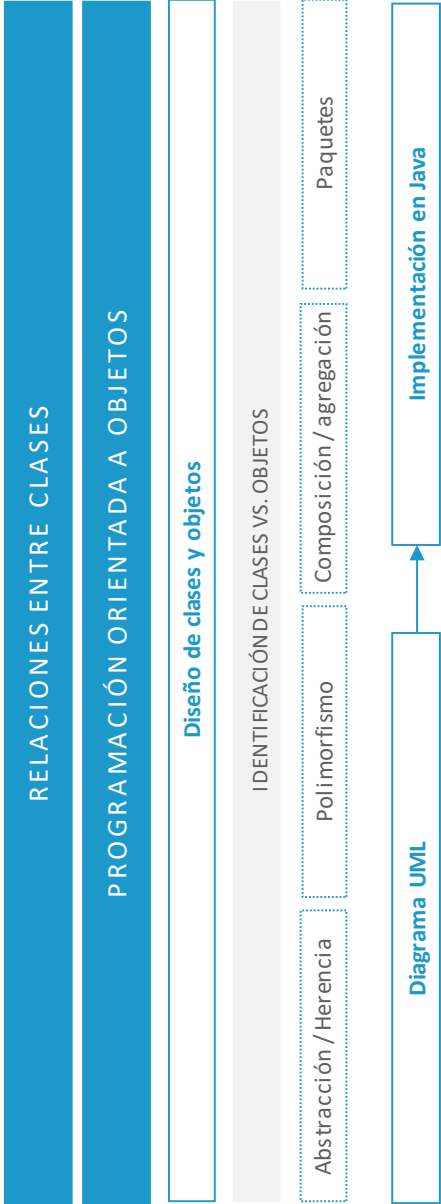


Programación Avanzada

Relaciones entre clases

Índice

Esquema	3
Ideas clave	4
2.1. Introducción y objetivos	4
2.2. Abstracción y herencia	4
2.3. Conceptos avanzados de herencia	9
2.4. Polimorfismo	9
2.5. Composición y agregación	11
2.6. This y super	13
A fondo	16
Actividades	21
Test	22



Esquema

2.1. Introducción y objetivos

Para estudiar este tema debes leer las páginas 75-114 y 123-124 del siguiente libro, disponible en la Biblioteca Virtual de UNIR:

García, L. F. (2010). *Todo lo básico que debería saber sobre programación orientada a objetos en Java* (pp. 75-114, 123-124). Barranquilla: Uninorte.

En este tema vamos a estudiar las principales relaciones entre clases. Entraremos en profundidad en todos los conceptos de diseño y modelado de diagramas de clases. A partir de estas relaciones podemos desarrollar soluciones a problemas complejos de programación.

- ▶ Identificar las relaciones entre clases en un problema.
- ▶ Identificar clases para diseñar soluciones con características tales como la reutilización y el encapsulamiento.

2.2. Abstracción y herencia

Abstracción

Según la RAE, es el proceso de separar por medio de una operación intelectual las cualidades de un objeto para considerarlas aisladamente o para considerar el mismo objeto en su pura esencia o noción.

En programación orientada a objetos hablamos de abstracción cuando en el **proceso de identificación de clases** seleccionamos aquellos **atributos y**

métodos comunes para formar una clase que nos permita **generalizar los valores comunes**.

Ejemplo:

Se quiere realizar un programa informático que gestione los empleados de una empresa informática. Existen dos tipos de empleados los empleados: internos y los colaboradores. Los empleados internos tienen nombre, apellidos, DNI, teléfono, cargo e ID de empleado. De los empleados externos se almacena el nombre, apellidos, DNI y empresa a la que pertenece.

La identificación de clases nos quedaría así:

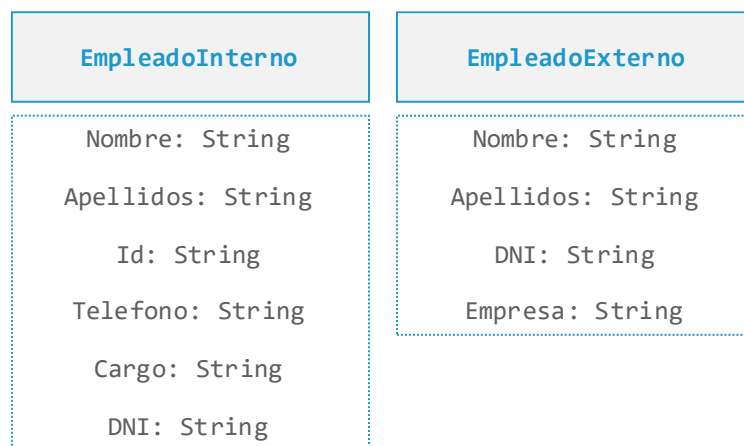


Figura 1. Clases de empleados.

Como podemos ver, hay **atributos** que son **comunes**. Si realizáramos la identificación de los **métodos** veríamos que ocurre lo mismo. De estas dos clases podríamos extraer una en la que se agruparan aquellos atributos y métodos comunes.

Con lo que nos quedaría algo similar a una clase en la que se agrupan todos los elementos comunes y luego dos clases que tienen la parte especializada.

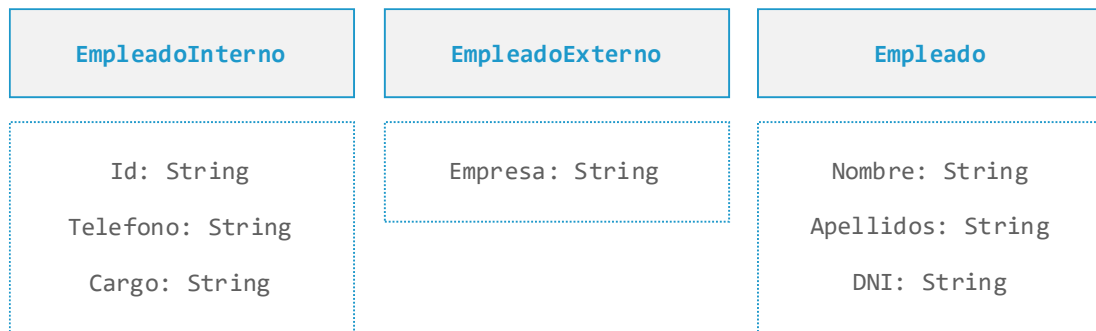


Figura 2. Atributos comunes y específicos.

Herencia

Es el proceso por el que una subclase obtiene todos los medios y atributos de una clase superior. Como se puede ver, la herencia no puede existir sin la abstracción.

Según el ejemplo anterior podemos decir que las clases EmpleadoInterno y EmpleadoExterno pueden heredar comportamiento de la clase Empleado.

En UML la herencia se identifica a través de una flecha que apunta a la clase padre.

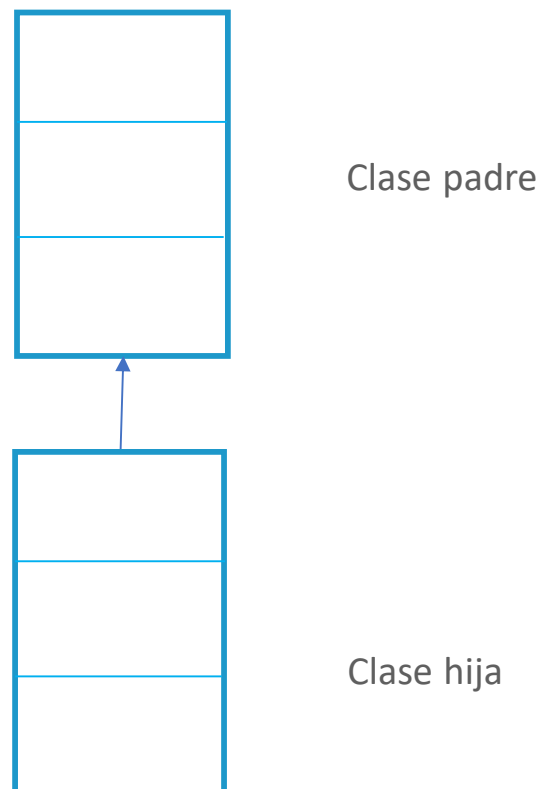


Figura 3. Herencia de clases.

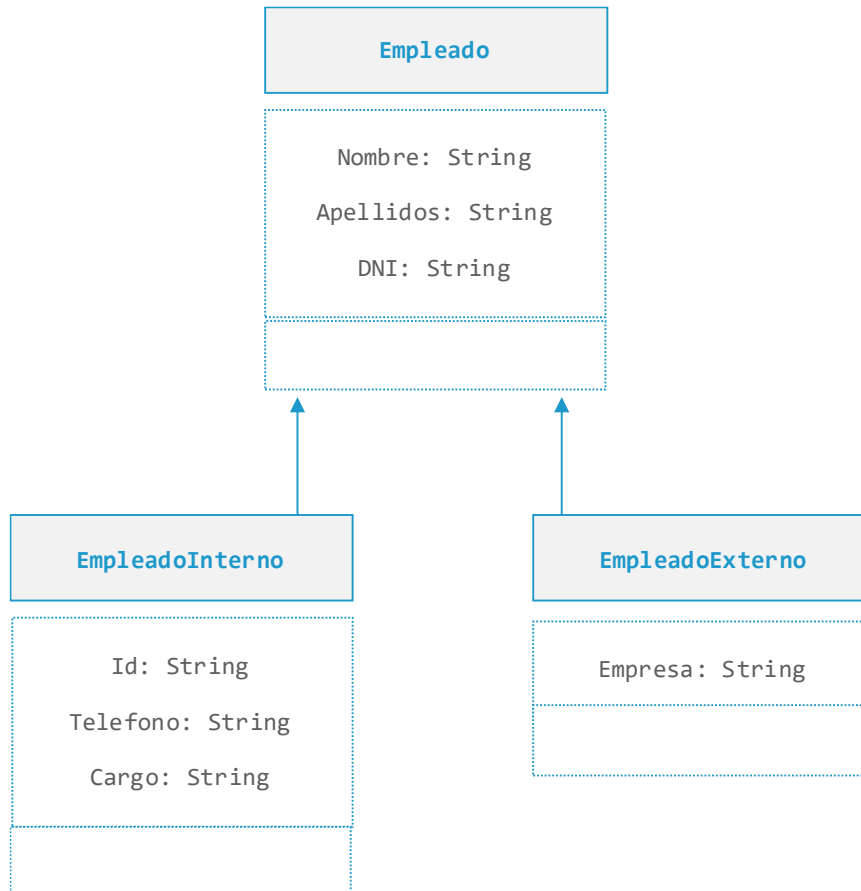


Figura 4. Herencia de clase II.

Implementación en Java

En Java la herencia se implementa a través de la palabra reservada `extends`.

```
class padre {  
}  
  
class hijo extends padre{  
}
```

Para acceder desde la clase hijo a la clase padre utilizamos la palabra reservada `super`. Si utilizamos `super()` o `super(argumentos)` lo que hacemos es invocar el método constructor de la clase padre.

La subclase:

- ▶ Hereda atributos y métodos.
- ▶ Siempre y cuando tengan los modificadores de accesibilidad adecuados (acceso paquete, public o protected).
- ▶ La subclase no tiene ningún miembro con el mismo nombre.
- ▶ Añade atributos y métodos.
- ▶ Modifica atributos y métodos.

Clase abstracta

Es aquella que no tiene implementado alguno de los métodos que la dotan de comportamiento. No es instanciable. Es decir, no se puede hacer new de la clase.

Cuando tenemos una clase abstracta pura, es decir, que no tiene implementados ninguno de sus métodos, hablamos de una interface. Para que una clase use una interface se utiliza la palabra reservada implements.

```
abstract class Figura{
    abstract int CalcularArea(){}
```



```
}
```



```
Interface Instrumento{
    Void tocar(){}
```



```
}
```



```
Class viento implements instrumento{
```



```
}
```


2.3. Conceptos avanzados de herencia

Herencia múltiple

En POO la herencia múltiple se refiere al proceso por el que una clase hereda de dos clases superiores.

En **Java** la herencia múltiple solo es posible si se realiza a partir de **interfaces**. Es posible combinar varias interfaces en una clase para simular la herencia múltiple.

2.4. Polimorfismo

Polimorfismo

Según la RAE es la cualidad de lo que tiene o puede tener distintas formas.

El polimorfismo dentro de la programación orientada a objetos es la capacidad que tiene un objeto de **ejecutar un método que cambia de comportamiento** dependiendo de cómo se haya realizado la instanciación del mismo.

La base para poder realizar polimorfismo es la sobreescripción de métodos. Se realiza a través de la anotación `@override`.

Una clase hereda el comportamiento de la clase base, pero reescribe los métodos necesarios para dotarla de otro comportamiento.

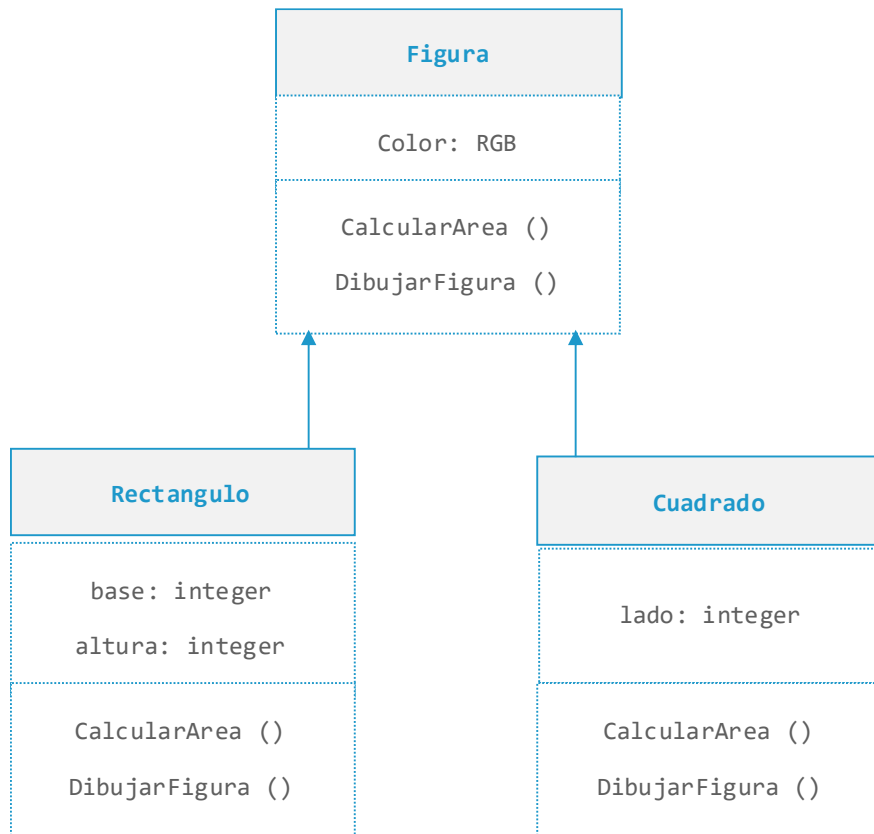


Figura 5. Ejemplo de polimorfismo.

En este caso, las figuras Rectángulo y Cuadrado heredan de la clase base Figura el atributo color. Los métodos `CalcularArea()` y `DibujarFigura()` los sobrescriben, ya que son diferentes dependiendo de la figura.

Para saber si un objeto corresponde a una clase se utiliza el operador `instanceof`.

Redefinición y sobrecarga

La redefinición es reescribir un método que se ha heredado con los mismos parámetros y sobrecargar un método es implementar varios métodos con el mismo nombre, pero diferentes parámetros. No se pueden redefinir métodos final ni métodos de clase (`static`).

La **ligadura** (conexión de un método con su llamada) se puede hacer en tiempo de ejecución.

- ▶ Ligadura temprana (métodos normales, sobrecargados, final).
 - En tiempo de compilación y enlace, el compilador necesita saber la referencia sobre la que se aplica el método.
- ▶ Ligadura tardía (polimorfismo).
 - La referencia al objeto que llama al método se hace en tiempo de ejecución.
 - No se sabe inicialmente el tipo de objeto que utiliza la llamada.

2.5. Composición y agregación

Agregación

Es un tipo de asociación que indica que una clase es parte de otra más grande, también se llama como composición débil. La destrucción del objeto compuesto no supone la destrucción de los objetos que la componen.

La agregación la utilizamos cuando queremos mostrar una dependencia entre objetos relacionados.

En UML se representa a través de un rombo en el contenedor.

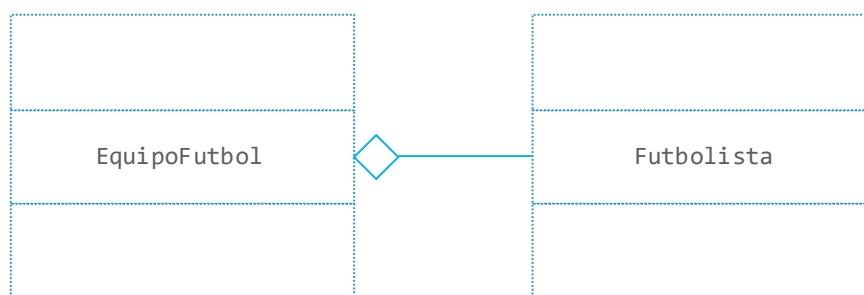


Figura 6. Agregación.

En este ejemplo, la destrucción del objeto que represente EquipoFutbol no implicaría la destrucción de los Futbolistas. De esta manera los objetos Futbolista seguirían viviendo a pesar de que el contenedor haya desaparecido.

Composición

Este tipo de asociación indica que una clase contenedora contiene elementos más **pequeños**. Los componentes forman parte del **objeto compuesto**. La **destrucción** del objeto contenedor supone la **eliminación de los objetos que la componen**. También se llama **agregación fuerte**.

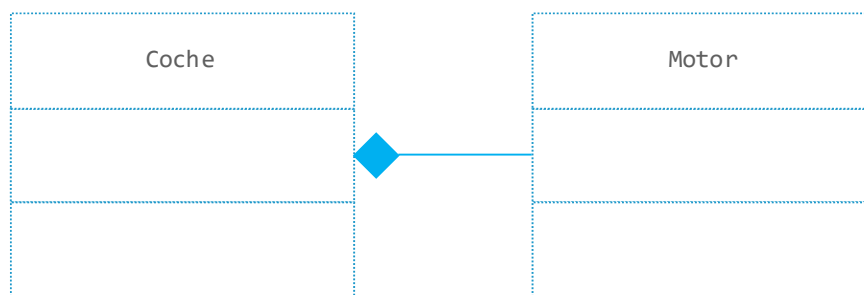


Figura 7. Composición.

En este caso lo que estamos indicando es que el objeto motor no puede vivir si desaparece el objeto coche. Ambos acabarían su ciclo de vida a la vez.

Para implementar la relación de composición en Java, se declaran los miembros de la clase compuesta como instancias de clase componedora.

Por ejemplo, una clase MiLinea, quedaría definida por dos objetos de la clase MiPunto, que representasen el punto de inicio y de fin de la línea:

```
class MiLinea {  
    MiPunto punto1;  
    MiPunto punto2;  
    MiLinea() {  
        punto1=new MiPunto();  
        punto2=new MiPunto();  
    }  
}
```

2.6. This y super

La palabra reservada `this` nos permite acceder a los miembros de la propia clase. Mediante `this.NombreVariable` estamos indicando que la variable `NombreVariable` es la de la propia clase.

```
public class Potencia {  
    double base;  
    int exponente;  
    double valor;  
  
    public Pontencia(double base, int exponente) {  
  
        this.base = base;  
        this.exponente = exponente;  
  
        if (exponente == 0) return;  
  
        for (;exponent>0;exponent--)  
            valor*=base;  
    }  
}
```

En el constructor de la clase `Potencia` los argumentos tienen el mismo nombre que las variables de la clase. En la sentencia `this.base = base`, `this.base` se refiere a la variable `base` de la clase `Potencia` y `base` se refiere al argumento del constructor.

La palabra reservada `super` nos permite hacer referencia al método o variable de la clase padre.

```
public class ClasePadre {  
    int i;  
    public ClasePadre() {  
        i = 10;  
    }  
}
```

```

        public setValor( int valor ) {
            i = valor;
        }
    }

    class ClaseHija extends ClasePadre {
        int j;

        public ClaseHija() {
            j = 12;
        }

        public setValor( int valor ) {
            j = valor/2;
            super.setValor( valor );
        }
    }

```

El constructor ClaseHija invoca por omisión al constructor sin argumentos de ClasePadre.

El método setValor de ClaseHija invoca al método setValor de ClasePadre utilizando la referencia super.

En el siguiente código, el constructor de la clase hija establecerá el valor de i a 10 y de j a 15, después cambiará el valor de j a 10 y el de i a 20:

```

ClaseHija ch;
ch = new ClaseHija();
ch.setValor( 20 );

```

Si realizamos la siguiente modificación en nuestras clases de ejemplo:

```

public class ClasePadre {
    int i;
    public ClasePadre() {

```

```

        i = 10;
    }

    public ClasePadre(int valor) {
        i = varlor;
    }

    public setValor( int valor ) {
        i = valor;
    }
}

class ClaseHija extends ClasePadre {
    int j;

    public ClaseHija() {
        j = 12;
    }

    public ClaseHija(int valor) {
        super(valor); //llamar al constructor de la superclase
        j = valor;
    }

    public setValor( int valor ) {
        j = valor/2;
        super.setValor( valor );
    }
}

```

En el siguiente código, el constructor de la clase hija establecerá el valor de i y de j a 30, después cambiará el valor de j a 10 y el de j a 20:

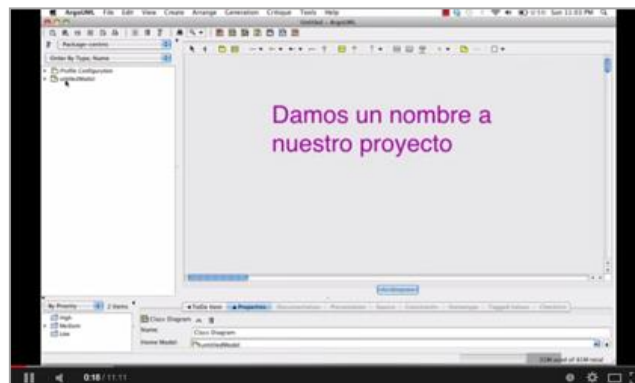
```

ClaseHija ch;
ch = new ClaseHija(30);
ch.setValor( 20 );

```

Proyecto en Argo UML

Vídeo que explica cómo crear un proyecto en ArgoUML.

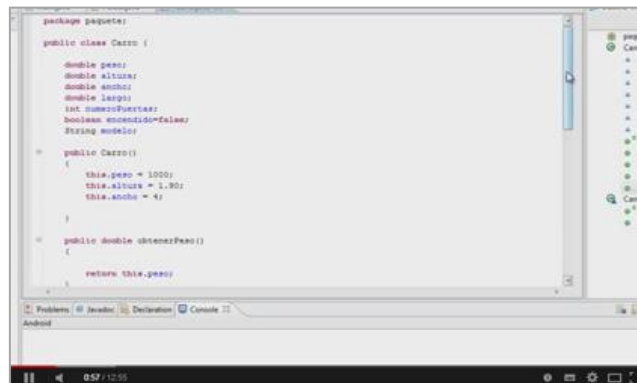


Accede al vídeo desde el aula virtual o a través de la siguiente dirección web:

<https://www.youtube.com/watch?v=BGM1QbPb9dQ>

Herencia en Java

Vídeo que explica mediante un ejemplo la herencia en Java.

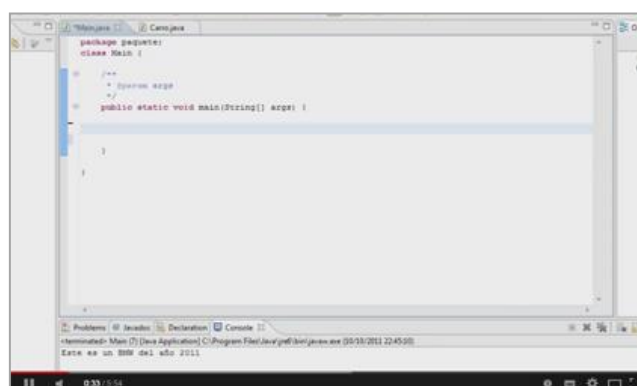


Accede al vídeo desde el aula virtual o a través de la siguiente dirección web:

<https://www.youtube.com/watch?v=CYNLhg42O9c>

Polimorfismo en Java

Vídeo que explica mediante un ejemplo el polimorfismo en Java.



Accede al vídeo desde el aula virtual o a través de la siguiente dirección web:

<https://www.youtube.com/watch?v=BSw1MLEc4PQ>

Paquetes en Java

Oracle (2019). Lesson: Packages. *The Java Tutorials*.

Consulta la página de Oracle en la que podrás encontrar información detallada sobre los paquetes en Java.



Accede al artículo desde el aula virtual o a través de la siguiente dirección web:

<http://docs.oracle.com/javase/tutorial/java/package/>

Herencia en Java

Oracle (2019). Inheritance. *The Java Tutorials*.

Consulta la página de Oracle en la que podrás encontrar información detallada sobre herencia en Java.



Accede al artículo desde el aula virtual o a través de la siguiente dirección web:

<http://docs.oracle.com/javase/tutorial/java/landl/subclasses.html>

Ejemplo de herencia en Java y sus limitaciones

Álvarez, C. (15 de agosto de 2017). Java Herencia y sus limitaciones. *Arquitectura Java*.



Ejemplo en el que se explica la herencia en Java y sus limitaciones.

Accede al artículo desde el aula virtual o a través de la siguiente dirección web:

<https://www.arquitecturajava.com/java-herencia-y-sus-limitaciones/>

Ejemplo de polimorfismo en Java

Moya, R. (18 de abril de 2014). Polimorfismo en Java (Parte I), con ejemplos. *Jarroba*.



Ejemplo en el que se explica el polimorfismo en Java (con ejemplos).

Accede al artículo desde el aula virtual o a través de la siguiente dirección web:

<https://jarroba.com/polimorfismo-en-java-parte-i-con-ejemplos/>

Herencia

García, L. F. (2010). *Todo lo básico que debería saber sobre programación orientada a objetos en Java*. Barranquilla: Uninorte.

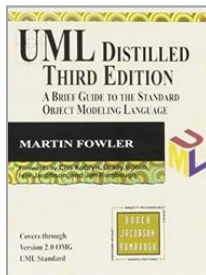


Las páginas 59-74 de este libro recogen varios ejemplos de implementación de la herencia en Java.

Accede al libro a través de la Biblioteca Virtual de UNIR

UML Distilled

Fowler, M. (2004). *UML Distilled: A Brief Guide to de Standard Object Modeling Language*. Massachusetts: Addison Wesley.



Libro que nos explica los fundamentos de UML de manera sencilla y útil. El libro describe los principales diagramas y cómo se puede hacer uso de ellos para desarrollar aplicaciones.

Accede a una parte del libro desde el aula virtual o a través de la siguiente dirección web:

<http://books.google.es/books?id=nHZsISr1gJAC&printsec=frontcover>

Programación orientada a objetos usando Java

Flórez, H. A. (2012). *Programación orientada a objetos usando Java*. Colombia: Ecoe Ediciones.



Manual en el que se hace una introducción a la programación orientada objetos con Java.

Accede al libro a través de la Biblioteca Virtual de UNIR

Laboratorio #1: Diseño e implementación de clases

Presentación del laboratorio

Para poder afrontar los ejercicios planteados en este laboratorio es necesario haber estudiado previamente los temas 1 y 2. Además, tendrás que tener instalado el entorno de desarrollo Eclipse, del que encontrarás más información en el apartado «Recursos externos» del tema.

Descripción del laboratorio

Este laboratorio está destinado a que pongas en práctica el diseño e implementación de clases en Java. Además, se trabajará el diagrama de clases en UML.

Entrega de laboratorio

La entrega de laboratorio deberá contener:

- ▶ Diagrama de clase en notación UML.
- ▶ Código fuente de la implementación.
- ▶ Memoria del desarrollo de la aplicación.

1. En Java la herencia se implementa:
 - A. Indicando en la clase padre los hijos que derivan de ella.
 - B. Indicando en el paquete las clases padre.
 - C. Indicando en la clase hija el padre de la que deriva.
 - D. Indicando en la definición del proyecto la jerarquía.

2. En Java la herencia múltiple:
 - A. No existe, pero se puede simular.
 - B. No existe y no se puede implementar.
 - C. Existe y se implementa igual que la herencia simple.
 - D. Existe y se implementa a través de una sentencia especial.

3. La agregación es:
 - A. La propiedad por la que una clase hereda comportamiento de otra clase llamada clase padre.
 - B. Es la relación que tienen dos clases que se crean de manera consecutiva.
 - C. Es la relación que tienen dos clases en las que una clase forma parte de la otra.
 - D. Es la relación que tienen dos clases que se intercambian mensajes.

4. Para acceder a una clase desde su clase hija utilizamos:
 - A. El operador `super`.
 - B. El operador `this`.
 - C. No se puede.
 - D. Instanciando la clase padre.

5. Para saber si un objeto se ha creado a partir de otra se utiliza:
- A. El operador `super`.
 - B. El operador `this`.
 - C. Se comparan con el `==`
 - D. Se utiliza el operador `instanceof`.
6. Las clases abstractas en Java:
- A. Se utilizan para implementar comportamientos especiales.
 - B. Se utilizan para crear objetos especiales que pueden ser serializados.
 - C. No pueden ser instanciadas.
 - D. Se pueden instanciar con un operador especial.
7. En UML la herencia se representa:
- A. Con un triángulo apuntando a la clase hija.
 - B. Con un triángulo apuntando a la clase padre.
 - C. Con un rombo en la clase padre.
 - D. Con un rombo en la clase hija.
8. En UML la composición se representa:
- A. Con un triángulo apuntando a la clase contenedora.
 - B. Con un triángulo apuntando a la clase contenida.
 - C. Con un rombo apuntando a la clase contenedora.
 - D. Con un rombo apuntando a la clase contenida.
9. Los paquetes en Java se utilizan:
- A. Para agrupar clases por comportamiento o función.
 - B. Para definir el comportamiento de las clases.
 - C. Para instanciar los objetos de manera más controlada.
 - D. Para aislar los programas del resto de ejecuciones.

10. Una clase que hereda de otra:

- A. Ve todos los métodos y atributos independientemente de su visibilidad.
- B. Ve todos los métodos y atributo menos los privados y protected.
- C. Ve todos los métodos y atributos menos los privados.
- D. Solo ve los métodos y atributos públicos.